

High-Precision Temperature Measurement using PT100: A Comparative Model Analysis

Puni Aditya, Vivek K Kumar and Dr. G.V.V. Sharma
Department of Electrical Engineering,
Indian Institute of Technology Hyderabad,
Kandi, India 502284
gadepall@ee.iith.ac.in

Abstract—This paper details the design, implementation, and comparative analysis of multiple calibration models for a temperature measurement system. The system utilizes a PT100 resistance temperature detector (RTD), a high-precision ADS1115 ADC, and an Arduino microcontroller. Two distinct experimental setups are presented. The first employs a Wheatstone bridge, the ADS1115 ADC, and a Least Squares (LSQ) model of the form $V = f(T)$ on a low-voltage dataset. The second experiment, using a simpler circuit and the Arduino’s internal 10-bit ADC, compares a Stochastic Gradient Descent (SGD) model and an Inverse LSQ model, both of the form $T = f(V)$, on a high-voltage dataset. The models are validated against unseen data, and their performance is compared to determine the most effective calibration strategy.

I. INTRODUCTION

Accurate temperature measurement is critical in various scientific and industrial applications. While many sensors exist, platinum resistance thermometers like the PT100 offer high accuracy and stability. However, their small resistance change necessitates precise measurement techniques. This project explores two hardware and three software approaches to this problem.

The first experiment (Exp. 1) aims for maximum hardware precision by using a Wheatstone bridge to linearize the sensor response and a 16-bit ADS1115 ADC to capture the small signal. A quadratic model of the form $V = f(T)$ is then fitted.

The second experiment (Exp. 2) tests a simpler, more software-centric approach. It uses a simpler circuit with the Arduino’s internal 10-bit ADC and applies more advanced machine learning models (SGD and Inverse LSQ) of the form $T = f(V)$ to achieve high accuracy from simpler hardware. This paper presents both hardware setups, their respective software, and a comparative analysis of all three mathematical models.

II. EXPERIMENT 1: WHEATSTONE BRIDGE WITH ADS1115

A. Hardware Setup (Exp. 1)

This experiment uses a Wheatstone bridge to isolate the small change in resistance from the PT100. The bridge consists of the 100Ω PT100, a 100Ω precision resistor, and two 4kΩ resistors. The output is fed into the ADS1115 ADC.

The components are listed in Table I. The complete circuit schematic is shown in Fig. 1.

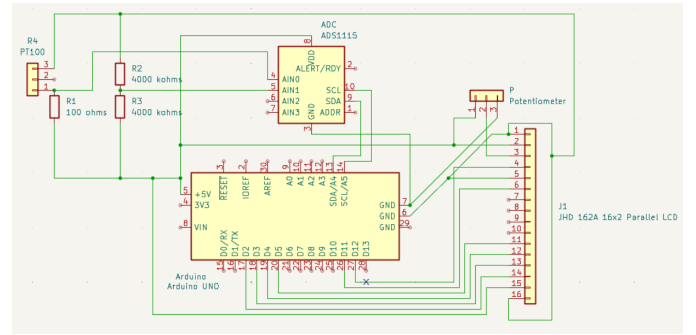


Fig. 1. Circuit schematic of the Experiment 1 system, featuring a Wheatstone bridge and the ADS1115 ADC.

TABLE I
LIST OF COMPONENTS (EXP. 1)

Component	Qty.	Description
Arduino Uno	1	Microcontroller for processing and control.
ADS1115 ADC	1	16-bit external ADC for high-precision voltage measurement.
PT100 RTD Sensor	1	Platinum resistance thermometer.
100Ω Resistor	1	Precision resistor for Wheatstone bridge.
4kΩ Resistors	2	Resistors for Wheatstone bridge.
JHD 162A Parallel LCD	1	For displaying the final temperature.
10kΩ Potentiometer	1	To adjust the contrast of the LCD.
Breadboard	1	For creating solderless circuit connections.
Jumper Wires	Set	For connecting all the components.

B. Role of the ADS1115 ADC

The ADS1115 is a high-precision, 16-bit Analog-to-Digital Converter.

- **High Resolution:** The 16-bit (65,536 steps) ADS1115 detects the microvolt-level changes from the bridge, which the Arduino’s 10-bit (1,024 steps) ADC would miss.
- **Differential Measurement:** The code reads the voltage difference between bridge output terminals (A0 and A1).

TABLE II
ADS1115 AND ARDUINO CONNECTIONS (EXP. 1)

ADS1115 Pin	Arduino Pin	Purpose
VDD	5V	Power Supply
GND	GND	Common Ground
SCL	A5 (SCL)	I2C Clock Line
SDA	A4 (SDA)	I2C Data Line
A0	-	Input from Voltage Divider Node (between PT100 and R_REF)
A1	Junction between the two 4kΩ resistors	Reference for differential reading

TABLE III
JHD 162A PARALLEL LCD AND ARDUINO CONNECTIONS

LCD Pin	Arduino Pin
VSS	GND
VDD	5V
V0	Potentiometer Middle Pin
RS	Digital 12
RW	GND
E	Digital 11
D4	Digital 5
D5	Digital 4
D6	Digital 3
D7	Digital 2
A (Anode)	5V
K (Cathode)	GND

This `ads.readADC_Differential_0_1()` function is crucial for rejecting common-mode noise.

- **Programmable Gain:** The code sets the internal amplifier gain to 8x (GAIN_EIGHT) using `ads.setGain()`, amplifying the small signal to use more of the ADC's range.

C. Software & Modeling (Exp. 1)

The Arduino code ('code.cpp', Listing ??) implements the model derived from 'lsq.py'. This model is a quadratic fit of the form $V = aT^2 + bT + c$, based on the training data in Table IV.

The derived coefficients are:

- $a = 1.7826 \times 10^{-5}$
- $b = 1.4987 \times 10^{-3}$
- $c = 0.1570$

To convert the measured voltage (V) back to temperature (T), the Arduino code solves the quadratic formula:

$$T = \frac{-b + \sqrt{b^2 - 4a(c - V)}}{2a}$$

This model was validated against a separate dataset (Table V), with results in Table VI.

TABLE IV
TRAINING DATA: EXPERIMENT 1 (LOW VOLTAGE)

Temp (°C)	Voltage (V)	Temp (°C)	Voltage (V)
26.7	0.20171	72.0	0.35171
36.7	0.23471	79.1	0.37671
43.9	0.26471	85.3	0.39771
47.8	0.28671	87.5	0.43271
56.6	0.29541	90.0	0.44671
59.1	0.30671	91.4	0.45271
60.0	0.30821	94.6	0.45871

TABLE V
VALIDATION DATA: EXPERIMENT 1 (LOW VOLTAGE)

Temp (°C)	Voltage (V)
36.8	0.23571
93.1	0.45391
57.3	0.29921
73.4	0.35621

TABLE VI
VALIDATION RESULTS: LSQ MODEL (EXP. 1)

Actual Temp (°C)	Voltage (V)	Predicted Temp (°C)
36.8	0.23571	36.64
93.1	0.45391	92.83
57.3	0.29921	57.17
73.4	0.35621	73.20

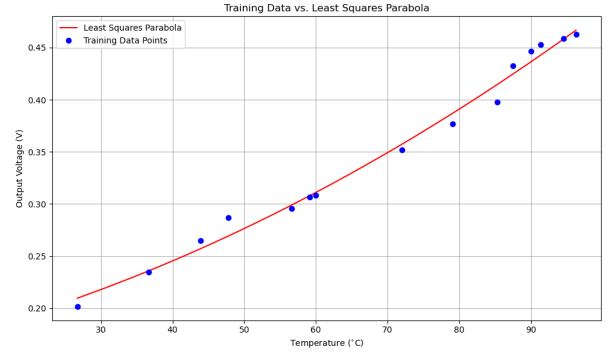


Fig. 2. Exp. 1: Training data vs. LSQ parabola.

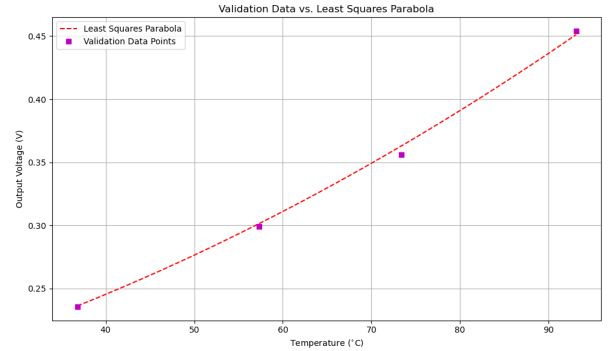


Fig. 3. Exp. 1: Validation data vs. LSQ parabola.

III. EXPERIMENT 2: INTERNAL ADC WITH DIRECT $T=F(V)$ MODELS

A second, simpler experiment was conducted to test "direct" models that predict Temperature from Voltage. This setup used a simpler circuit (likely a voltage divider) and the Arduino's built-in 10-bit ADC (`analogRead()`). This resulted in a higher voltage range (~ 2.6 - 2.9 V). The training and validation data for this experiment are shown in Tables VII and VIII.

TABLE VII
TRAINING DATA: EXPERIMENT 2 (HIGH VOLTAGE)

Temp (°C)	Voltage (V)	Temp (°C)	Voltage (V)
32.1	2.66	71.8	2.79
36.6	2.67	76.6	2.81
41.1	2.68	81.3	2.82
45.5	2.69	85.9	2.83
50.8	2.71	90.9	2.85
56.8	2.74	95.9	2.87
62.1	2.75	97.6	2.89
67.6	2.77		

TABLE VIII
VALIDATION DATA: EXPERIMENT 2 (HIGH VOLTAGE)

Temp (°C)	Voltage (V)
32.0	2.66
36.5	2.67
41.0	2.68
45.5	2.69
50.5	2.71
56.6	2.74
62.0	2.75
67.5	2.77
71.5	2.79
76.5	2.81
81.1	2.82
85.9	2.82
90.7	2.85
95.5	2.87
97.5	2.89

A. Model 2: Stochastic Gradient Descent (SGD)

This approach employs a two-part machine learning workflow. First, an offline training script (`model_trainer.py`) uses a 2nd-degree polynomial feature set, feature scaling, and a Stochastic Gradient Descent Regressor (`SGDRegressor`) to learn a baseline model from a static dataset. The complete processing pipeline (polynomial transformer, scaler, and model) is then persisted using `joblib`.

This method uses `PolynomialFeatures` to create V and V^2 terms, and `StandardScaler` to normalize these features. Normalization ensures that all features contribute appropriately to the gradient descent, leading to a more stable model. The resulting equation, while complex, is implemented directly on the Arduino (Listing ??). A key feature of this method is the ability to use `partial_fit` for "online learning," allowing the model to adapt to new data in real-time and compensate for sensor drift, a significant advantage over static calibration methods.

B. Model 3: Inverse Least Squares

As a direct comparison to the SGD model, a standard LSQ regression was performed on the same high-voltage dataset, but with the model inverted to $T = aV^2 + bV + c$. This provides a direct, non-scaled polynomial fit. The coefficients found are:

- $a \approx 269.84$
- $b \approx -953.11$
- $c \approx 874.19$

C. Model Comparison

To visualize all three models and both datasets, the data from Experiment 1 was artificially scaled by adding a voltage offset. Figure 4 shows all data points and all three predictive models on a single graph. Validation results for the Experiment 2 models are in Tables IX and X.

TABLE IX
VALIDATION RESULTS: SGD MODEL (EXP. 2)

Actual Temp (°C)	Voltage (V)	Predicted Temp (°C)
32.0	2.66	31.89
36.5	2.67	36.46
41.0	2.68	41.00
45.5	2.69	45.50
50.5	2.71	50.41
56.6	2.74	56.59
62.0	2.75	62.13
67.5	2.77	67.56
71.5	2.79	71.74
76.5	2.81	76.60
81.1	2.82	81.33
85.9	2.82	81.33
90.7	2.85	90.57
95.5	2.87	95.04
97.5	2.89	97.35

TABLE X
VALIDATION RESULTS: INVERSE LSQ MODEL (EXP. 2)

Actual Temp (°C)	Voltage (V)	Predicted Temp (°C)
32.0	2.66	31.81
36.5	2.67	36.33
41.0	2.68	40.83
45.5	2.69	45.30
50.5	2.71	50.14
56.6	2.74	56.12
62.0	2.75	62.00
67.5	2.77	67.75
71.5	2.79	72.24
76.5	2.81	77.34
81.1	2.82	81.79
85.9	2.82	81.79
90.7	2.85	90.38
95.5	2.87	95.96
97.5	2.89	99.27

IV. CONCLUSION AND FINAL RECOMMENDATION

This paper successfully designed and tested a PT100-based thermometer using two different hardware setups and three calibration models.

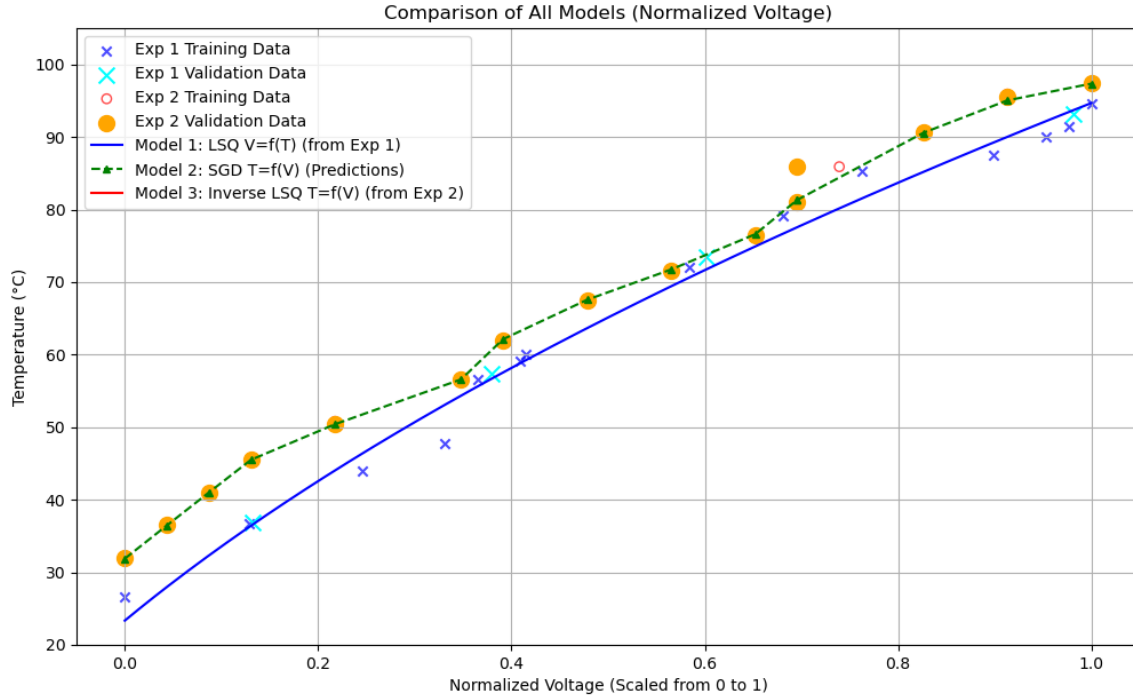


Fig. 4. Comparison of all models. Exp 1 data is scaled by an offset (approx. +2.4V) to be plotted on the same axis as Exp 2 data for visualization.

- **Model 1 (LSQ ' $V=f(T)$ ')**: This model, paired with the ADS1115 and Wheatstone bridge, is hardware-intensive but precise. It requires solving a quadratic equation on the microcontroller.
- **Model 2 (SGD ' $T=f(V)$ ')**: This model, paired with the internal ADC, is complex to derive (requiring a ML pipeline) but results in a simple (though long) equation on the Arduino.
- **Model 3 (Inverse LSQ ' $T=f(V)$ ')**: This model uses the same simple hardware as Model 2 but is the simplest to derive and implement, providing a direct $T = aV^2 + bV + c$ formula.

Conclusion: Comparing the validation results from Experiment 2 (Tables IX and X), both the SGD and Inverse LSQ models perform exceptionally well, with predictions very close to the actual temperatures. The ****Inverse Least Squares model (Model 3) is arguably the best method****, as it provides nearly identical accuracy to the complex SGD pipeline while being far simpler to calculate, debug, and implement.

The hardware from Experiment 1 (Wheatstone bridge + ADS1115) is the most robust and accurate.

Therefore, the optimal system would be a combination of the superior hardware from Experiment 1 (Wheatstone bridge, ADS1115) and the simpler, direct modeling approach from Experiment 2 (an Inverse LSQ model trained on the low-voltage data).